

Mutual Exclusion and
Asynchronous Completion
CS 111
Fall 2024
Operating System Principles
Peter Reiher

Outline

- Mutual exclusion
- Asynchronous completions

Mutual Exclusion

- Critical sections can cause trouble when more than one thread executes them at a time
 - Each thread doing part of the critical section before any of them do all of it
- Preventable if we ensure that only one thread can execute a critical section at a time
- We need to achieve *mutual exclusion* of the critical section
 - If one thread is running the critical section, the other definitely isn't

Critical Sections in Applications

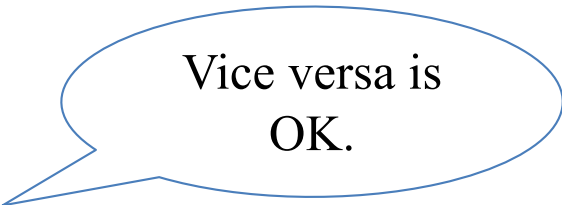
- Most common for multithreaded applications
 - Which frequently share data structures
- Can also happen with processes
 - Which share operating system resources
 - Like files
 - Or multiple related data structures
- Avoidable if you don't share resources of any kind
 - But that's not always feasible

Recognizing Critical Sections

- Generally involves updates to object state
 - May be updates to a single object
 - May be related updates to multiple objects
- Generally involves multi-step operations
 - Object state inconsistent until operation finishes
 - Pre-emption compromises object or operation
- Correct operation requires mutual exclusion
 - Only one thread at a time has access to object(s)
 - Client 1 completes before client 2 starts

Critical Sections and Atomicity

- Using mutual exclusion allows us to achieve *atomicity* of a critical section
- Atomicity has two aspects:
 1. Before or After atomicity
 - A enters critical section before B starts
 - B enters critical section after A completes
 - There is no overlap
 2. All or None atomicity
 - An update that starts will complete or will be undone
 - An uncompleted update has no effect
- Correctness generally requires both



Vice versa is OK.

Options for Protecting Critical Sections

- Turn off interrupts
 - We covered that in the last lecture
- Avoid shared data whenever possible
- Protect critical sections using hardware mutual exclusion
 - In particular, atomic CPU instructions
- Software locking

Not available for applications, limits all concurrency

Often impossible

Also often impossible

Avoiding Shared Data

- A good design choice when feasible
- Don't share things you don't need to share
- But not always an option
- Even if possible, may lead to inefficient resource use
- Sharing read only data also avoids problems
 - If no writes, the order of reads doesn't matter
 - But a single write can blow everything out of the water

Atomic Instructions

- CPU instructions are uninterruptable
- What can they do?
 - Read/modify/write operations
 - Can be applied to 1-8 contiguous bytes
 - Simple: increment/decrement, and/or/xor
 - Complex: test-and-set, exchange, compare-and-swap
- Can we do entire critical section in one instruction?
 - With careful design, some data structures can be implemented this way

Usually not feasible

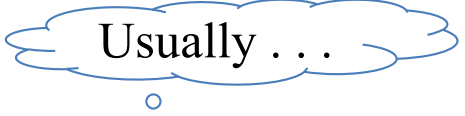
- Doesn't help with waiting synchronization

Locking

- Protect critical sections with a data structure
- Locks
 - The party holding a lock can access the critical section
 - Parties not holding the lock cannot access it
- A party needing to use the critical section tries to acquire the lock
 - If it succeeds, it goes ahead
 - If not . . . ?
- When finished with critical section, release the lock
 - Which someone else can then acquire

Software Locks

- ISAs usually do not include instructions for completely building locks
 - Individual instructions are properly serialized
 - But multiple instructions are not
- So we need to build locks in software
 - Leading to issues of enforcing their mutual exclusion
 - Which have different solutions in different cases



Usually . . .

Using Locks

- Remember this example?

thread #1

thread #2

counter = counter + 1; counter = counter + 1;

*What looks like one instruction in C
gets compiled to:*

mov counter, %eax

add \$0x1, %eax

mov %eax, counter

Three instructions . . .

- How can we solve this with locks?

Using Locks For Mutual Exclusion

```
pthread_mutex_t lock;  
pthread_mutex_init(&lock, NULL);  
...  
if (pthread_mutex_lock(&lock) == 0) {  
    counter = counter + 1;  
    pthread_mutex_unlock(&lock);  
}
```



Now the three assembly instructions are mutually exclusive

How Do We Build Locks?

- The very operation of locking and unlocking a lock is itself a critical section
 - If we don't protect it, two threads might acquire the same lock
- Sounds like a chicken-and-egg problem
- But we can solve it with hardware assistance
- Individual CPU instructions are atomic
 - So if we can implement a lock with one instruction
- . . .

Using Single Instructions To Build Locks

- Sounds tricky
- The core operation of acquiring a lock (when it's free) requires:
 1. Check that no one else has it
 2. Change something so others know we have it
- Sounds like we need to do two things in one instruction
- No problem – hardware designers have provided for that

Atomic Instructions – Test and Set

A C description of a machine language

instruction **REAL Instructions are silicon, not C!!!**

```
bool TS( char *p) {  
    bool rc;  
    rc = *p;           /* note the current value          */  
    *p = TRUE;         /* set the value to be TRUE          */  
    return rc;         /* return the value before we set it */  
}
```

```
if !TS(flag) {  
    /* We have control of the critical section! */  
}
```

If rc was false,
nobody else ran
TS. We got the
lock!

If rc was true,
someone else already
ran TS. They got the
lock!

Atomic Instructions – Compare and Swap

Again, a C description of machine instruction

```
bool compare_and_swap( int *p, int old, int new ) {  
    if (*p == old) {          /* see if value has been changed      */  
        *p = new;             /* if not, set it to new value   */  
        return( TRUE);        /* tell caller he succeeded     */  
    } else                    /* someone else changed *p    */  
        return( FALSE);       /* tell caller he failed      */  
}  
  
if (compare_and_swap(flag,UNUSED,IN_USE) {  
    /* I got the critical section! */  
} else {  
    /* I didn't get it.  */  
}
```

Using Atomic Instructions to Implement a Lock

- Assuming silicon implementation of test and set

```
bool getlock( lock *lockp) {  
    if (TS(lockp) == 0 )  
        return( TRUE);  
    else  
        return( FALSE);  
}  
void freelock( lock *lockp ) {  
    *lockp = 0;  
}
```

Note that building the lock requires more than the TS instruction.

Lock Enforcement

- Locking resources only works if either:
 - It's not possible to use a locked resource without the lock
 - Or everyone who might use the resource carefully follows the rules
- Otherwise, a thread might use the resource when it doesn't hold the lock
- We'll return to practical options for enforcement later

What Happens When You Don't Get the Lock?

- You could just give up
 - But then you'll never execute your critical section
- You could try to get it again
- But it still might not be available
- So you could try to get it yet again . . .

Spin Waiting



- Spin waiting for a lock is called *spin locking*
- The computer science equivalent
- Check if the event occurred
- If not, check again
- And again
- And again
- . . .

Spin Locks: Pluses and Minuses

- Good points:
 - Properly enforces access to critical sections
 - Assuming properly implemented locks
 - Simple to program
- Dangers:
 - Wasteful
 - Spinning uses processor cycles
 - Likely to delay freeing of desired resource
 - The cycles burned could be used by the locking party to finish its work
 - Bug may lead to infinite spin-waits

The Asynchronous Completion Problem

- Parallel activities move at different speeds
- One activity may need to wait for another to complete
- The *asynchronous completion problem* is:
 - How to perform such waits without killing performance?
- Examples of asynchronous completions
 - Waiting for an I/O operation to complete
 - Waiting for a response to a network request
 - Delaying execution for a fixed period of real time
- Can we use spin locks for this synchronization?

Spinning Sometimes Makes Sense

1. When awaited operation proceeds in parallel
 - A hardware device accepts a command
 - Another core releases a briefly held spin lock
2. When awaited operation is guaranteed to happen soon
 - Spinning is less expensive than sleep/wakeup
3. When spinning does not delay awaited operation
 - Burning CPU delays running another process
 - Burning memory bandwidth slows I/O
4. When contention is expected to be rare
 - Multiple waiters greatly increase the cost

Yield and Spin

- Check if your event occurred
- Maybe check a few more times
- But then yield
- Sooner or later you get rescheduled
- And then you check again
- Repeat checking and yielding until your event is ready

Problems With Yield and Spin

- Extra context switches
 - Which are expensive
- Still wastes cycles if you spin each time you're scheduled
- You might not get scheduled to check until long after event occurs
- Works very poorly with multiple waiters
 - Potential unfairness

Fairness and Mutual Exclusion

- What if multiple processes/threads/machines need mutually exclusive access to a resource?
- Locking can provide that
- But can we make guarantees about fairness?
- Such as:
 - Anyone who wants the resource gets it sooner or later (no starvation)
 - Perhaps ensuring FIFO treatment
 - Or enforcing some other scheduling discipline

How Can We Wait?

- Spin locking/busy waiting
- Yield and spin ...
- Either spin option may still require mutual exclusion
 - And any time spent spinning is wasted
- And fairness may be an issue
- *Completion events*

Completion Events

- If you can't get the lock, block
- Ask the OS to wake you when the lock is available
- Similarly for anything else you need to wait for
 - Such as I/O completion
 - Or another process to finish its work
- Implemented with *condition variables*

Condition Variables

- Create a synchronization object associated with a resource or request
- Initially set to “event hasn’t happened”
- Requester blocks and is queued awaiting event on that object
 - I.e., “Event has happened”
- Upon completion, the event is “posted”
 - Posting event to object unblocks one or more waiters for that event
 - Depending . . .
- Return condition variable to “event hasn’t happened” (if others might wait on it)

Condition Variables and the OS

- Generally the OS provides condition variables
 - Or library code that implements threads does
- Block a process or thread when a condition variable is used
 - Moving it out of the ready queue
- It observes when the desired event occurs
- It then unblocks the blocked process or thread
 - Putting it back in the ready queue
 - Possibly preempting the running process

Handling Multiple Waits

- Threads will wait on several different things
- Pointless to wake up everyone on every event
 - Each should wake up only when his event happens
- So OS (or thread package) should allow easy selection of “the right one”
 - When some particular event occurs
- But what if several threads are waiting for the same thing?

Waiting Lists

- Suggests each completion event needs an associated waiting list • • •
 - When posting an event, consult list to determine who's waiting for that event
 - Then what?
 - Wake up everyone on that event's waiting list?
 - One-at-a-time in FIFO order?
 - One-at-a-time in priority order (possible starvation)?
 - Choice depends on event and application

This isn't the ready queue!

Who To Wake Up?

- Who wakes up when a condition variable is signaled?
 - `pthread_cond_wait` ... at least one blocked thread
 - `pthread_cond_broadcast` ... all blocked threads
- The broadcast approach may be wasteful
 - If the event can only be consumed once
 - Potentially unbounded waiting times
- A waiting queue would solve these problems
 - Each post wakes up the first client on the queue

Locking and Waiting Lists

- Spinning for a lock is usually a bad thing
 - Locks should probably have waiting lists
- A waiting list is a (shared) data structure
 - Implementation will likely have critical sections
 - Which may need to be protected by a lock
- This seems to be a circular dependency
 - Locks have waiting lists
 - Which must be protected by locks
 - What if we must wait for the waiting list lock?
 - Where does it end?



Process A
Process B
Process C



Process D
Process E
Process F



?????

A Real Problem For Waiting Lists

- The sleep/wakeup race condition

Consider this sleep code:

```
void sleep( eventp *e ) {
    while(e->posted == FALSE) {
        add_to_queue( &e->queue,
            myproc );
        myproc->runstate |= BLOCKED;
        yield();
    }
}
```

And this wakeup code:

```
void wakeup( eventp *e) {
    struct proce *p;

    e->posted = TRUE;
    p = get_from_queue(&e->
        queue);
    if (p) {
        p->runstate &= ~BLOCKED;
        resched();
    } /* if !p, nobody's
        waiting */
}
```

What's the problem with this?

A Sleep/Wakeup Race

- Let's say thread B has locked a resource and thread A needs to get that lock
- So thread A will call `sleep()` to wait for the lock to be free
- Meanwhile, thread B finishes using the resource
 - So thread B will call `wakeup()` to release the lock
- No other threads are waiting for the resource

The Race At Work

Thread A

```
void sleep( eventp *e ) {  
    while(e->posted == FALSE) {
```

CONTEXT SWITCH!

Nope, nobody's in the queue!

CONTEXT SWITCH!

```
        add_to_queue( &e->queue, myproc );  
        myproc->runstate |= BLOCKED;  
        yield();  
    }  
}
```

Thread B

The event hasn't happened yet!

```
void wakeup( eventp *e) {  
    struct proce *p;    Now it happens!  
  
    e->posted = TRUE;  
    p = get_from_queue(&e-> queue);  
    if (p) {  
  
        } /* if !p, nobody's waiting */  
    }
```

The effect?

Thread A is sleeping

But there's no one to
wake him up

Solving the Problem

- There is clearly a critical section in `sleep()`
 - Starting before we test the posted flag
 - Ending after we put ourselves on the notify list and blocked° ○ ○
 - Think about why these actions are part of the critical section.
- During this section, we need to prevent:
 - Wakeups of the event SEE IF YOU CAN
 - Other people waiting on the event FIGURE THAT OUT FOR YOURSELF!
- This is a mutual-exclusion problem
 - Fortunately, we already know how to solve those
 - We just need a lock°.°.°
 - But how will we handle contention for that lock?

Conclusion

- Two classes of synchronization problems:
 1. Mutual exclusion
 - Only allow one of several activities to happen at once
 2. Asynchronous completion
 - Properly synchronize cooperating events
- Locks are one way to assure mutual exclusion
- Spinning and completion events are ways to handle asynchronous completions